

COURSE INTRODUCTION

ML-DL-Ops

From Lab to
Production

Prof. Mayank Vatsa

IIT Jodhpur • Department of CSE

The Uncomfortable Truth

87%

of ML projects never make it to production

— Gartner, VentureBeat Research

Why do models that work perfectly in notebooks fail in the real world?

Case Study: Our Deepfake Detection Journey

In the Lab: We built a deepfake detection model. High accuracy on benchmark datasets. Published papers. Everything worked perfectly.

DATA ISSUES

Formats, resolution, language, codecs



MODEL ISSUES

Batch inference, GPU memory



SYSTEM ISSUES

API, latency, scaling



SECURITY

Adversarial attacks

The Mission: Deploy for India's 2024 Elections — millions of users, real-time detection, adversaries actively trying to fool the system.

The Lesson: The model was **5% of the work**. Making it production-ready was the other **95%**. This is the gap MLOps bridges.

What Broke (And When)

FILE FORMATS

Trained on JPG

Users uploaded PNG, WebP, HEIC, BMP...

RESOLUTION

Trained on 256×256

Received 4K, thumbnails, WhatsApp compressed

LANGUAGE

Trained on English content

Hindi, regional languages, code-mixed

VIDEO CODECS

Trained on H.264

H.265, VP9, AV1, screen recordings

BATCH vs SINGLE

Trained with batch size 32

Production: one image at a time, different statistics

GPU MEMORY

Single request: 2GB

100 concurrent users = OOM crash

API DESIGN

REST API for images

Video streaming? Progress updates? Timeouts?

LATENCY

Model inference: 200ms

User expectation: instant. Network + preprocessing + postprocessing = 2s+

ELECTION SCALE

Lab: 100 images/day

Election day: 100,000+ requests/hour, spiky traffic

ADVERSARIAL ATTACKS

We detected adversarial inputs — people actively trying to fool our system

Each of these issues required engineering solutions — not ML solutions

Lab vs Production: The Reality Gap

IN THE LAB

- Clean, curated datasets
- Single-user environment
- Accuracy is the only metric
- One-time training
- Manual, ad-hoc processes
- Jupyter notebooks



IN PRODUCTION

- Messy, evolving data
- Millions of concurrent requests
- Latency, cost, fairness matter
- Continuous retraining
- Automated, reproducible pipelines
- Scalable infrastructure

This gap is what MLOps addresses

What is ML-DL-Ops?

MLOps is a set of practices that combines Machine Learning, DevOps, and Data Engineering to deploy and maintain ML systems in production reliably and efficiently.



Also known as: ML Engineering, Production ML, ML Systems

The ML Lifecycle: A Pipeline View



Key Insight: In traditional ML courses, we focus on steps 3-5. In production, **data management, deployment, and monitoring consume 80%+ of the effort.**

WHAT ML COURSES TEACH

Model architecture, loss functions, optimization, evaluation metrics

WHAT PRODUCTION NEEDS

Data pipelines, versioning, monitoring, scaling, security, compliance

This course bridges that gap

When ML Goes Wrong: Real Failures

DATA DRIFT

Zillow iBuying Disaster

ML model for home prices failed to adapt to market changes. Lost \$500M+, laid off 25% of workforce.

Root cause: No monitoring for distribution shift

BIAS & FAIRNESS

Amazon Hiring Tool

Resume screening AI learned to penalize women. Trained on 10 years of male-dominated hiring data.

Root cause: Biased training data, no fairness audits

DEPLOYMENT FAILURE

Knight Capital Crash

Faulty deployment of trading algorithm. Lost \$440M in 45 minutes due to untested code in production.

Root cause: No CI/CD, no rollback mechanism

Common Theme: These weren't model failures — they were **systems failures**. Models worked in testing but failed in production.

Evolution: DevOps → MLOps

DevOps

- Version control for **code**
- CI/CD for applications
- Infrastructure as Code
- Monitoring app metrics
- Automated testing

+

+ ML-Specific

- Version **data + models**
- Training + inference pipelines
- Feature stores & experiment tracking
- Model performance monitoring
- Data validation & drift detection

=

MLOps

- End-to-end automation
- Reproducible experiments
- Continuous training (CT)
- Model governance
- Responsible AI practices

MLOps extends DevOps principles to handle the unique challenges of ML systems

Key Challenges in Production ML

DATA MANAGEMENT

Quality, versioning, pipelines, privacy, lineage

REPRODUCIBILITY

Environment parity, experiment tracking, artifacts

SCALABILITY

Distributed training, serving at scale, resources

MODEL DECAY

Data drift, concept drift, degradation over time

DEPLOYMENT

Containerization, orchestration, A/B testing

MONITORING

Performance tracking, alerting, observability

LATENCY & COST

Inference optimization, GPU costs, compression

GOVERNANCE

Model registry, approvals, audit trails, compliance

ETHICS & FAIRNESS

Bias detection, explainability, responsible AI

This course addresses each of these challenges systematically

Your Learning Journey

1 Foundations of ML/DL Systems

Data to Deployment Pipeline, Lab-to-Production Failures, DevOps to MLOps

2 Infrastructure & Tools

Virtual Environments, Accelerators, Containers, Kubernetes, Reproducibility

3 Model Development & Optimization

Data Processing, Features, Training, Quantization, Distributed, vLLM, TensorRT

4 Deployment & Operations

Versioning, Monitoring, CI/CD Pipelines, Training vs Inference Pipelines

5 Performance

6 Advanced Topics

7 Ethics & Reliability

Course Project: End-to-End ML System

WHAT YOU'LL BUILD

A complete ML application from data ingestion to production deployment — with monitoring, versioning, and automated retraining.

Project Components

- Data pipeline with validation & versioning
- Feature store implementation
- Model training with experiment tracking
- Containerized deployment (Docker + K8s)
- CI/CD pipeline for model updates
- Monitoring dashboard with drift detection

EXAMPLE PROJECT DOMAINS

Computer Vision

Real-time object detection, image classification API

NLP / LLM Applications

Sentiment analysis, RAG system, text generation

Recommendation Systems

Personalized content, collaborative filtering

Time Series / Forecasting

Demand prediction, anomaly detection

Teams of 2-3 students • Incremental milestones throughout the semester

Tools & Technologies You'll Master

ML FRAMEWORKS

PyTorch, TensorFlow, Keras, JAX

EXPERIMENT TRACKING

MLflow, Weights & Biases, TensorBoard

DATA VERSIONING

DVC, Delta Lake, Git LFS

CONTAINERIZATION

Docker, Kubernetes, Helm

CI/CD & ORCHESTRATION

GitHub Actions, Airflow, Kubeflow

SERVING & INFERENCE

TorchServe, TensorRT, ONNX, vLLM

MODEL OPTIMIZATION

Quantization, Pruning, Distillation, Mixed Precision

MONITORING

Prometheus, Grafana, Evidently AI

CLOUD PLATFORMS

AWS SageMaker, GCP Vertex AI, Azure ML

Note: We'll primarily use open-source tools that work on local infrastructure, with optional cloud deployment exercises. Focus is on principles — tools change, concepts remain.

Assessment & Expectations

ASSESSMENT BREAKDOWN

35%

Course Project

End-to-end ML system with milestones

15%

Labs

10 hands-on lab assignments

10%

Mid-Sem

Covers Topics
Till Mid-Sem

40%

End-Sem

Comprehensive

PREREQUISITES

Machine Learning fundamentals

Python programming proficiency

Basic Linux/command line

Git version control basics

WHAT'S EXPECTED

Active participation in labs

Willingness to debug & troubleshoot

Collaboration on team projects

Hands-on mindset — learning by doing

Summary & Next Steps

KEY TAKEAWAYS

- 1 87% of ML projects fail to reach production — this course addresses why
- 2 MLOps bridges the gap between lab experiments and production systems
- 3 You'll build a complete end-to-end ML system as your course project
- 4 Focus on principles over tools — transferable skills for any platform

Questions?

BEFORE NEXT CLASS

Set up development environment (Python 3.10+, Git)

Install Docker Desktop on your laptop

Create accounts: GitHub, Docker Hub

Read: "Hidden Technical Debt in ML Systems"

NEXT LECTURE

Deep dive into the ML Pipeline: Data to Deployment challenges and architecture patterns